

ИТМО · ТЕСТИРОВАНИЕ ПО · ОСЕНЬ 2026

Роль тестирования в жизненном цикле ПО

Тестирование ПО · Модуль 1 · Лекция 2

Андрей Попов · Лектор

4 курс

ИТМО · Тестирование ПО

План лекции

01 Жизненный цикл ПО (SDLC):
модели и фазы

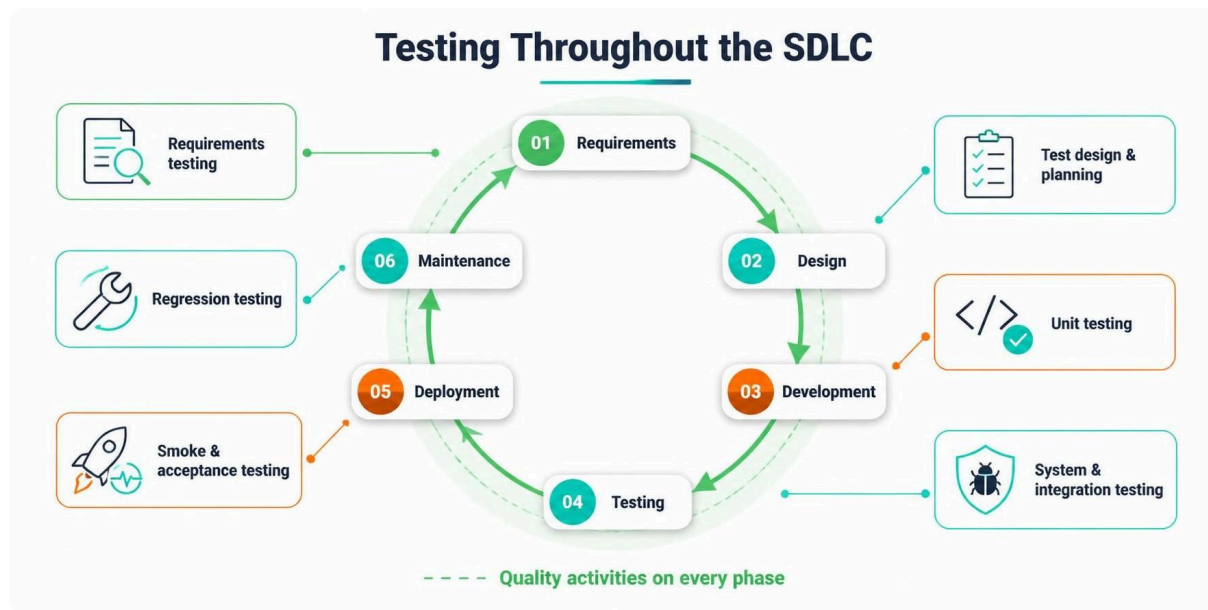
02 Где тестирование в каждой фазе

03 Процесс тестирования (STLC):
фазы и артефакты

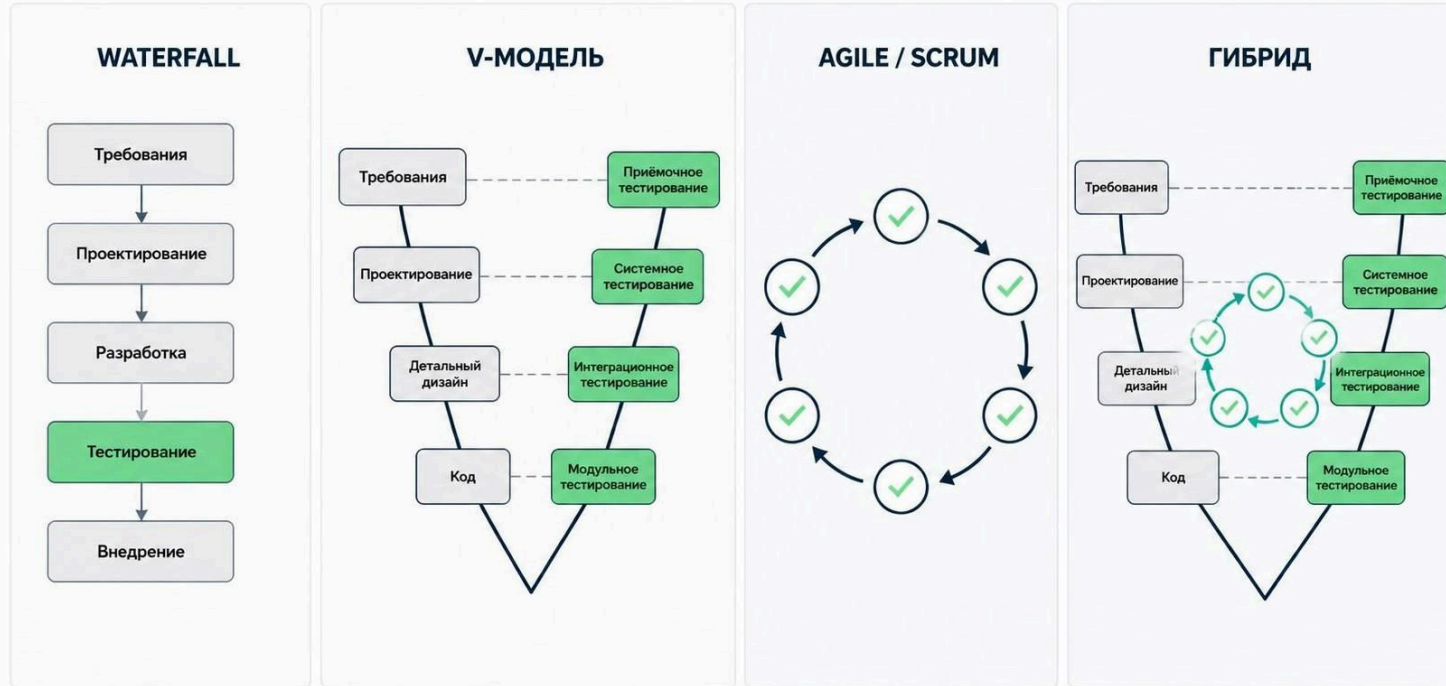
04 Роли в тестировании: QA, test
engineer, разработчик, аналитик

05 Релизная политика и подход к
тестированию: почему нельзя
всегда релизить за час

06 Модель данных СМП — основа
для тест-дизайна

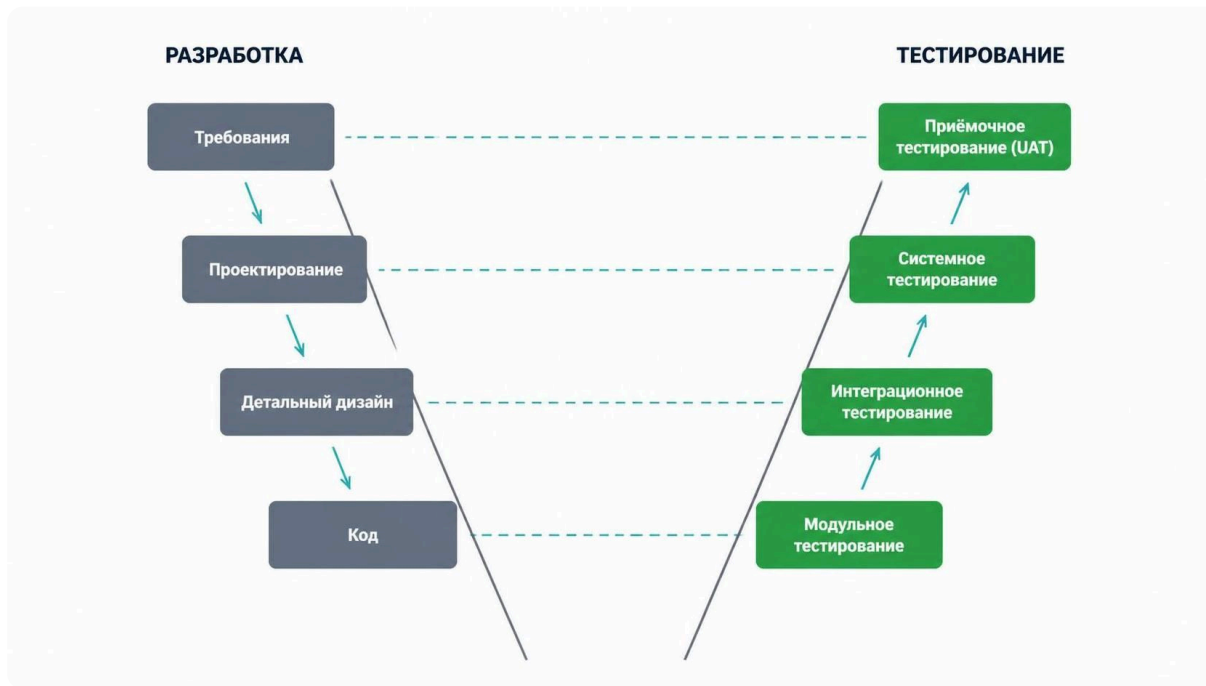


Тестирование — не одна фаза в конце, а деятельность на ВСЕХ фазах. Качество строится на каждом этапе, а не «проверяется» post factum.



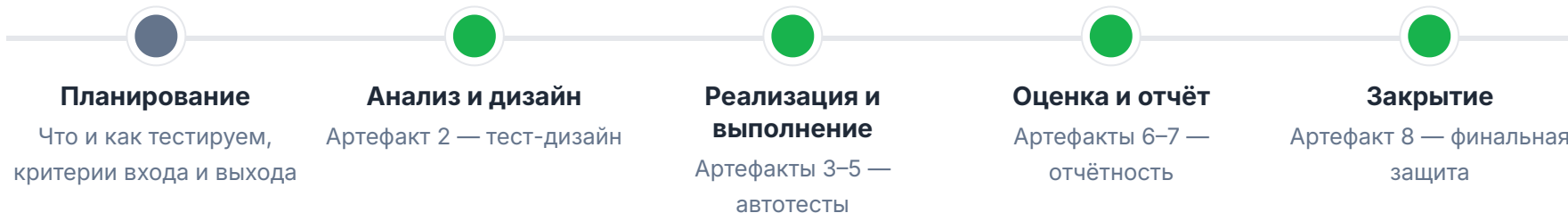
Где тестирование в каждой фазе SDLC

Фаза	Деятельность тестировщика
Требования	Анализ тестируемости, выявление противоречий, приёмочные критерии
Проектирование	Ревью архитектуры, план тестирования, оценка рисков
Разработка	Unit-тесты, статический анализ кода, code review
Тестирование	Интеграционные, системные, E2E, регрессионные тесты
Развёртывание	Smoke-тесты, canary-релизы, мониторинг
Поддержка	Регрессия при багфиксах, тестирование патчей



Каждому уровню разработки соответствует свой уровень тестирования — **нельзя заменить один уровень другим.**

Процесс тестирования (STLC)



В реальных проектах фазы **пересекаются и идут параллельно**, в Agile STLC сжат до спринта. В курсе вы проходите их **последовательно** — по одному артефакту на фазу.

Артефакты тестирования



Тест-план

Что, как, когда и кем тестируется; критерии начала и окончания



Тест-кейс

Предусловия, шаги, входные данные, ожидаемый результат



Баг-репорт

Заголовок, шаги воспроизведения, ожидание vs факт, severity, priority



Тест-отчёт

Что проверено, что найдено, какие риски остались, рекомендации



Тестовая стратегия

Уровни, инструменты, автоматизация vs ручное

Роли в тестировании



QA Engineer

Процессы,
стандарты, метрики
качества — уровень
организации



Test Engineer

Проектирование и
автотесты — уровень
продукта



Разработчик

Unit-тесты,
статический анализ,
code review



Аналитик

Требования и
приёмочные
критерии

Независимость: тестировщик не создаёт и не чинит дефекты — поэтому сообщает о проблемах без конфликта интересов.

→ SDET: тот же трек Test Engineer + инженерная глубина (фреймворки, tooling, CI).

Релизная политика определяет риск → риск определяет подход к тестированию



Прямой деплой (startup/SaaS)

- Один прод, одна конфигурация — вы контролируете инфраструктуру
- Релизить часто: smoke-тесты, canary-релизы
- Баг → хотфикс за полчаса



Вендорская модель (enterprise)

- Релиз уходит клиенту на установку
- Нужна гарантия качества ДО отгрузки
- Откатить чужой прод нельзя — тестирует сам клиент

Матрица тестирования вендора: ОС × СУБД × конфигурация × версия клиента

Lekton: 10 продов, 2–3 линейки в поддержке, бэкпорты — время релиза ограничено тестированием, а не сборкой.



Quality gates: например, coverage $\geq 70\%$, все тесты зелёные, нет критических багов.

В курсе: GitHub Actions · JaCoCo · Allure Reports

Реальность Lekton



Гибрид

V-модель для регуляtorики +
Agile для фич



CI/CD

На всех контурах, ~40
тестирующих, Java-стек



Вывод

Тестирование встроено в
каждый этап, а не «в конце»

Card

```
pan · bin · status · dailyLimit ·  
monthlyLimit · availableBalance ·  
expiryDate
```

```
status: ACTIVE | INACTIVE | BLOCKED |  
EXPIRED
```

Transaction

```
mti · stan · rrn · pan · amount · status ·  
declineReason · authCode
```

```
status: APPROVED | DECLINED
```

Ключевые поля для тест-дизайна: **status** (4 значения), **limits** (daily/monthly), **amount**, **expiryDate** — основа классов эквивалентности, границ и decision tables.

Ключевые принципы архитектуры СМП



Гибридное взаимодействие

Синхронный HTTP + асинхронный RabbitMQ



Eventual consistency

Запись в лог не мгновенно, а после доставки через очередь



Внешний API Bin Lookup

Timeout/retry — точка отказа



Каждый принцип

Порождает тестовые сценарии

Что будет на практике

Практика 1 (содержательная): запуск СМП + smoke-тесты → Артефакт 1 (6 баллов)

Практика 2 (проходная): приём артефакта 1 + разбор архитектуры СМП, карта требований

ДЗ к модулю 2: изучить T3 Authorization и Card Management

Подготовка к артефакту 2 (тест-дизайн)

Классы эквивалентности · границы · decision table

Вопросы?